

Walmart Business case study

Dataset link: walmart_data.csv

- User_ID: User ID
- Product_ID: Product ID
- Gender: Sex of User
- Age: Age in bins
- Occupation: Occupation
- City_Category: Category of the City (A,B,C)
- StayInCurrentCityYears: Number of years stay in current city
- Marital_Status: Marital Status
- ProductCategory: Product Category
- Purchase: Purchase Amount

Import the dataset and do usual data analysis steps like checking the structure & characteristics of the dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
df = pd.read_csv('/content/walmart_data.CSV')

# Display the first 5 rows to understand the data
print("First 5 rows of the dataset:")
display(df.head())
```

The screenshot shows a Google Colab notebook interface. The top part of the notebook contains code to load a dataset and display its first 5 rows. The output shows a table with 11 columns: User_ID, Product_ID, Gender, Age, Occupation, City_Category, Stay_In_Current_City_Years, Marital_Status, Product_Category, and Purchase. The first 5 rows of data are displayed. Below this, the notebook shows code to display the data types of all columns. The output shows the data types for each column.

```
# Load the dataset
df = pd.read_csv('/content/walmart_data.csv')

# Display the first 5 rows to understand the data
print("First 5 rows of the dataset:")
display(df.head())
```

First 5 rows of the dataset:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	7969

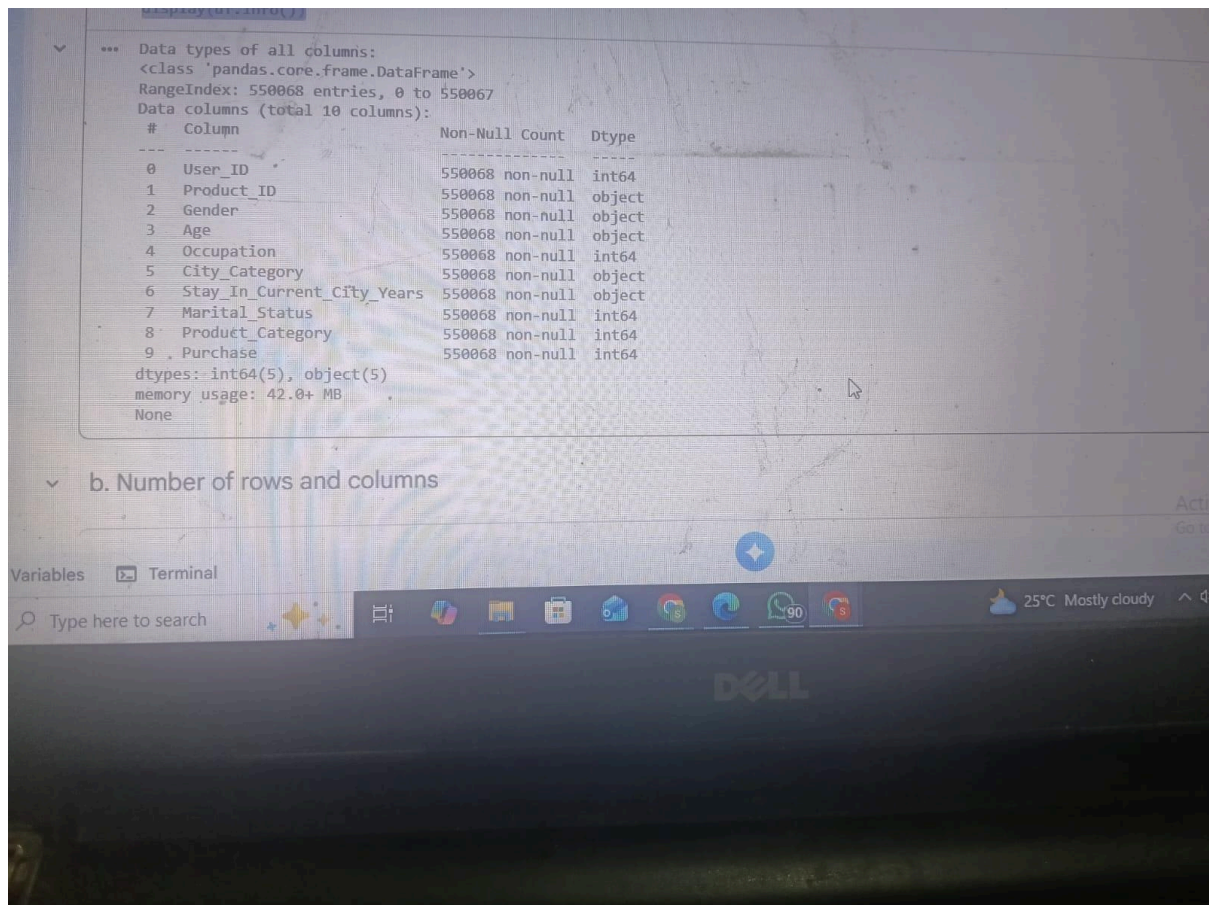
```
# Display data types
print("Data types of all columns:")
display(df.info())
```

Data types of all columns:

```

User_ID      object
Product_ID   object
Gender       object
Age          object
Occupation   object
City_Category object
Stay_In_Current_City_Years  int64
Marital_Status  int64
Product_Category  int64
Purchase      int64
dtype: object
```

```
# Display data types
print("Data types of all columns:")
display(df.info())
```



b. Number of rows and columns

```
# Display the shape of the dataset  
print(f"Number of rows: {df.shape[0]}")  
print(f"Number of columns: {df.shape[1]}")
```

Result

Number of rows: 550068
Number of columns: 10

c. Missing values in each column

```
print("Number of missing values per column:")  
display(df.isnull().sum())
```


Run all

Cannot save changes

11



```
# Check for missing values
```

```
print("Number of missing values per column")  
display(df.isnull().sum())
```



... Number of missing values per column:

0

User_ID

0

Product_ID

0

Gender

0

Age

0

Occupation

0

City_Category

0

Stay_In_Current_City_Years

0

Marital_Status

0

Product_Category

0

Purchase

0

dtype: int64

available

2 Detect Null values and outliers



DELL

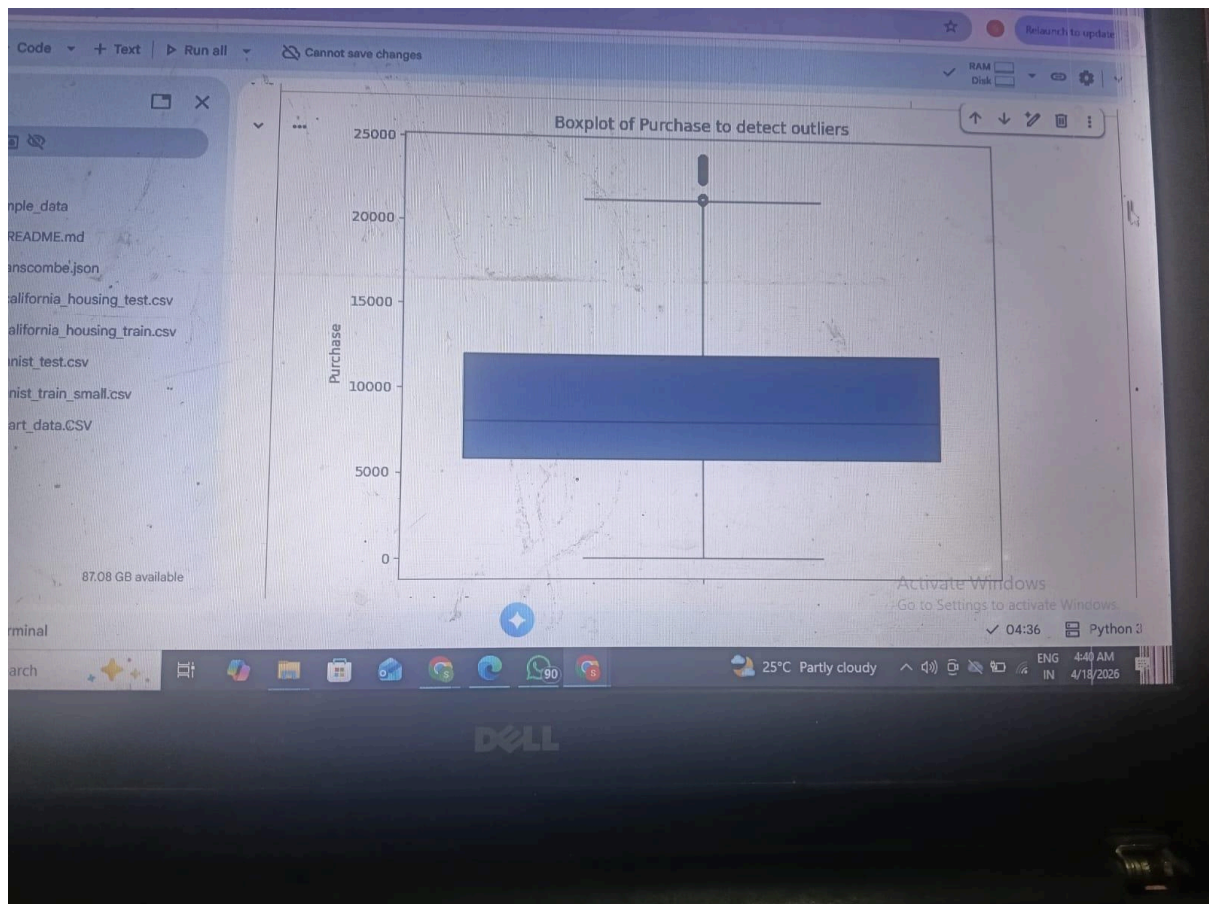
2. Detect Null values and outliers

a. Find the outliers for every continuous variable in the dataset

```
continuous_vars = ['Purchase']
```

```
plt.figure(figsize=(8, 6))
sns.boxplot(y=df[continuous_vars[0]])
plt.title(f'Boxplot of {continuous_vars[0]} to detect outliers')
plt.ylabel(continuous_vars[0])
plt.show()

print(f"Summary statistics for {continuous_vars[0]}:")
display(df[continuous_vars[0]].describe())
```



Result:

	Purchase
count	550068.000000
	0

mean	9263.968713
std	5023.065394
min	12.000000
25%	5823.000000
50%	8047.000000
75%	12054.000000
max	23961.000000

b. Remove/clip the data between the 5 percentile and 95 percentile

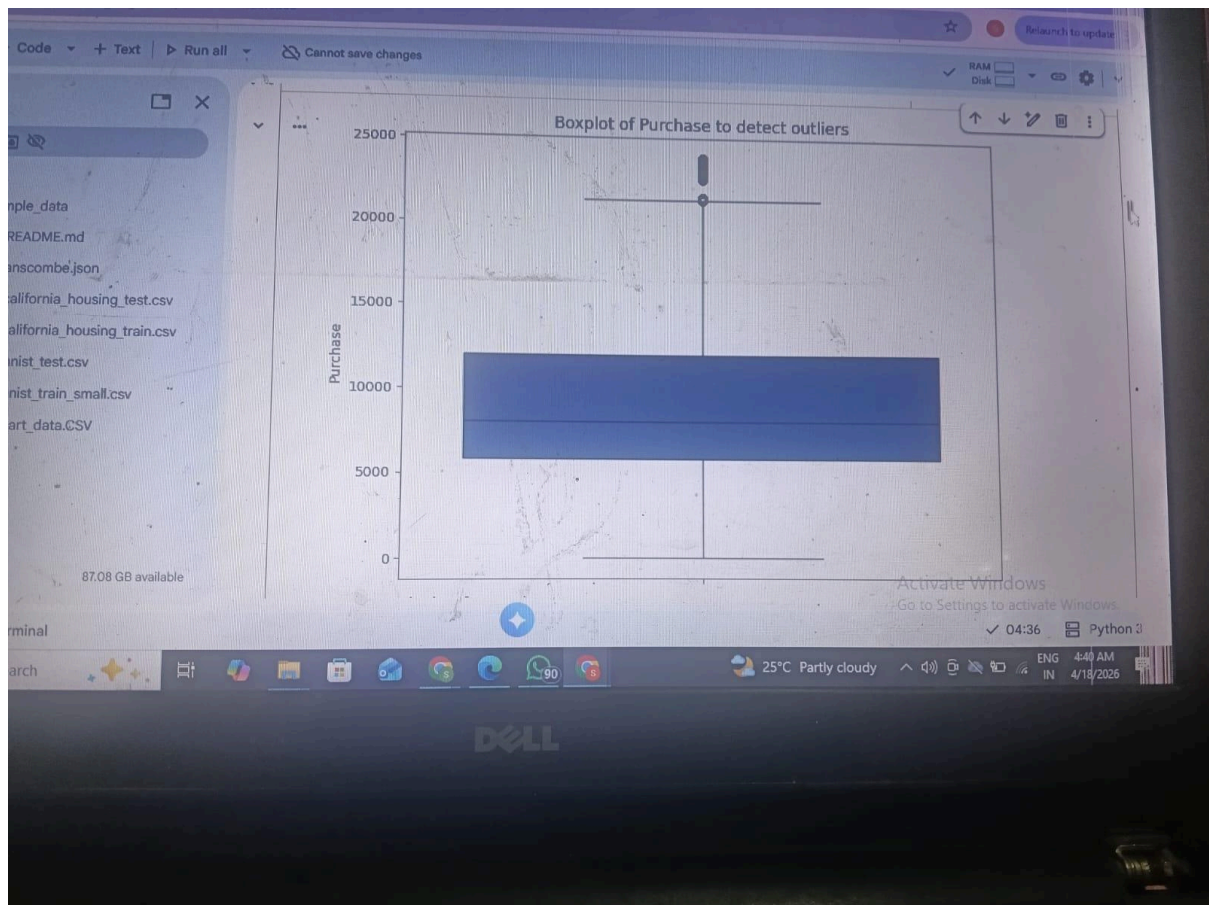
```
# Calculate 5th and 95th percentiles for 'Purchase'
pct_5 = df['Purchase'].quantile(0.05)
pct_95 = df['Purchase'].quantile(0.95)

# Clip the 'Purchase' column
df['Purchase_clipped'] = np.clip(df['Purchase'], pct_5, pct_95)

print(f"Original    'Purchase'    min:    {df['Purchase'].min()},    max:
{df['Purchase'].max()}")
print(f"5th percentile: {pct_5}, 95th percentile: {pct_95}")
print(f"Clipped  'Purchase' min:    {df['Purchase_clipped'].min()},    max:
{df['Purchase_clipped'].max()}")

# Display summary statistics for the clipped column
print("\nSummary statistics for clipped 'Purchase' column:")
display(df['Purchase_clipped'].describe())
```

Result:



Original 'Purchase' min: 12, max: 23961
5th percentile: 1984.0, 95th percentile: 19336.0
Clipped 'Purchase' min: 1984, max: 19336

Summary statistics for clipped 'Purchase' column:

	Purchase_clipped
count	550068.000000
mean	9256.710489
std	4855.947166
min	1984.000000
25%	5823.000000
50%	8047.000000
75%	12054.000000
max	19336.000000

3. Data Exploration

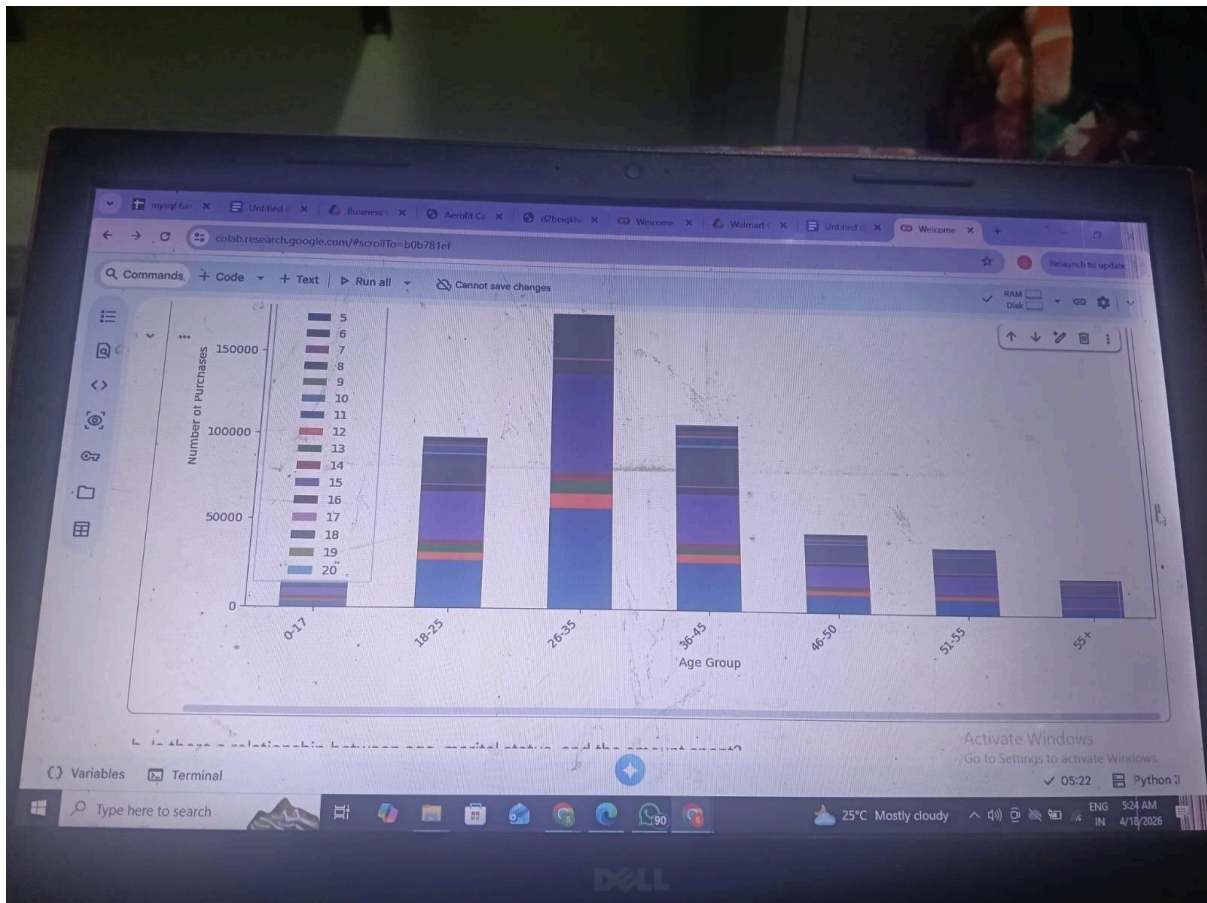
a. What products are different age groups buying?

```
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
import IPython.display as display

# Group by Age and Product_Category and count purchases
age_product_category_counts = df.groupby(['Age',
'Product_Category']).size().unstack(fill_value=0)

# Plotting a stacked bar chart to visualize product categories by age
group
fig, ax = plt.subplots(figsize=(12, 6), facecolor='w')
age_product_category_counts.plot(kind='bar', stacked=True, ax=ax)
ax.set_title('Product Categories Purchased by Age Group')
ax.set_xlabel('Age Group')
ax.set_ylabel('Number of Purchases')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()

data = io.BytesIO()
plt.savefig(data)
image =
F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Product Categories Purchased by Age Group"
display.display(display.Markdown(F"! [{alt}] ({image})"))
plt.close(fig)
```

b. Is there a relationship between age, marital status, and the amount spent?

```
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
import IPython.display as display

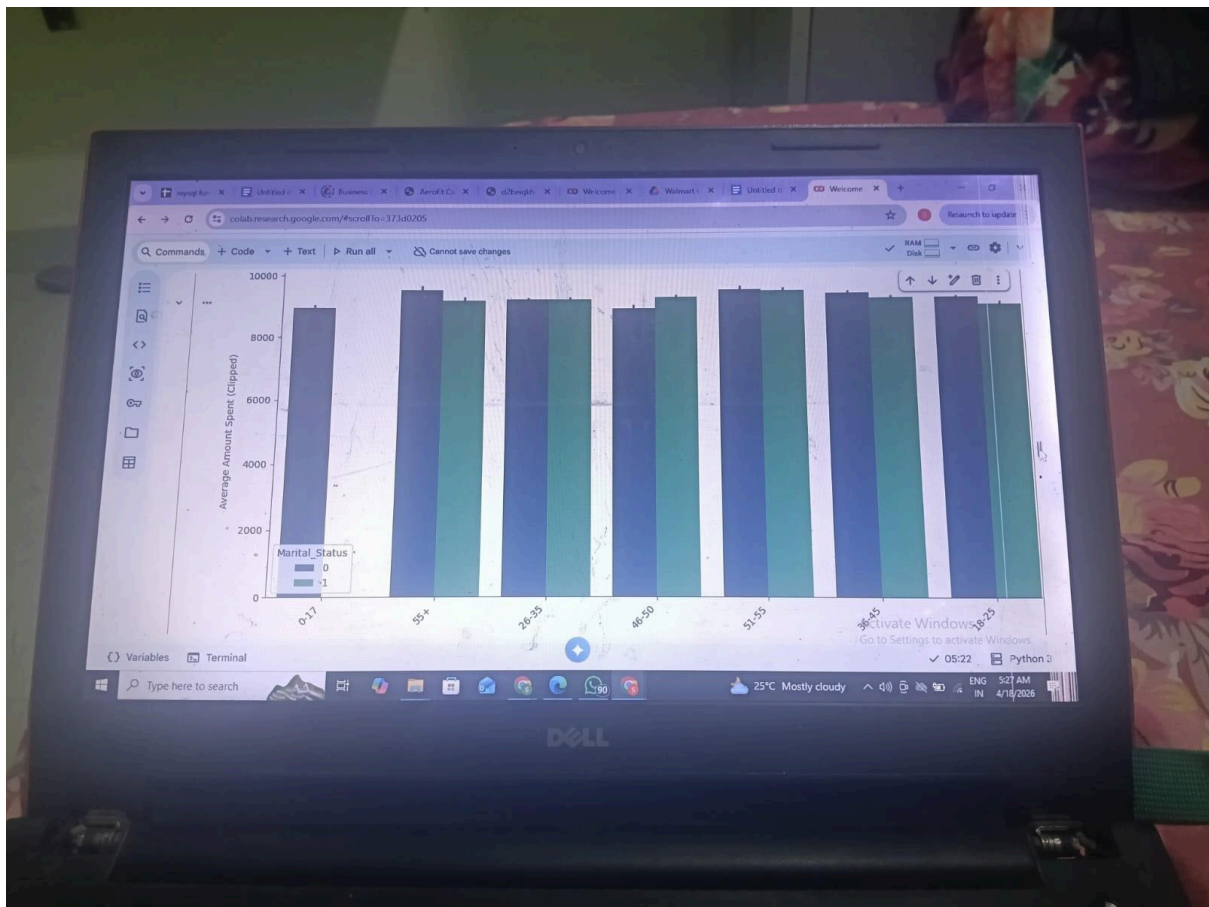
# Using 'Purchase_clipped' for amount spent as it handles outliers
fig, ax = plt.subplots(figsize=(12, 6), facecolor='w')
sns.barplot(data=df, x='Age', y='Purchase_clipped',
hue='Marital_Status', ax=ax, palette='viridis')
ax.set_title('Average Amount Spent by Age Group and Marital Status')
ax.set_xlabel('Age Group')
ax.set_ylabel('Average Amount Spent (Clipped)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
```

```

data = io.BytesIO()
plt.savefig(data)
image =
F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Average Amount Spent by Age Group and Marital Status"
display.display(display.Markdown(F"! [{alt}] ({image})"))
plt.close(fig)

```

Result:



c. Are there preferred product categories for different genders?

```

import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
import IPython.display as display

# Group by Gender and Product_Category and count purchases

```

```

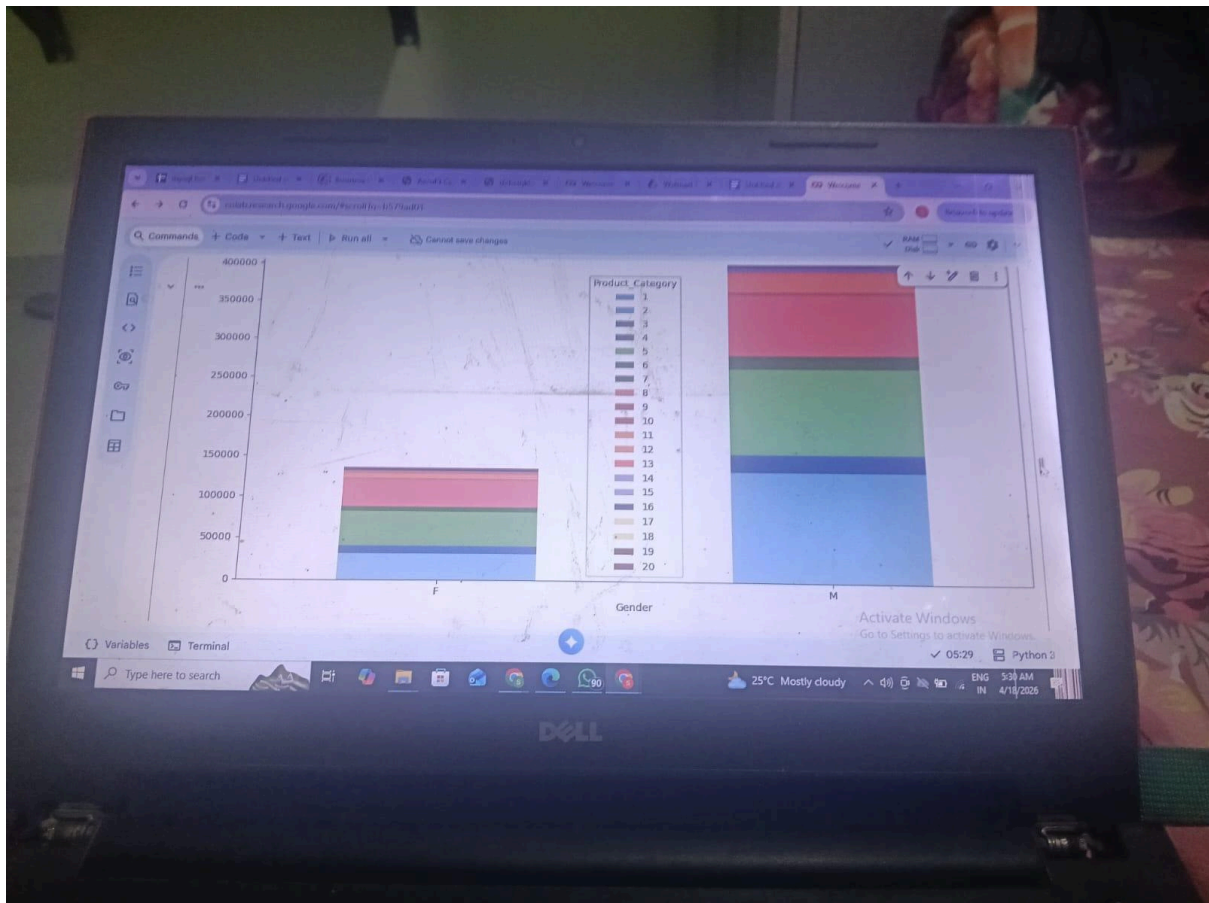
gender_product_category_counts = df.groupby(['Gender',
'Product_Category']).size().unstack(fill_value=0)

# Plotting a stacked bar chart to visualize product categories by
gender
fig, ax = plt.subplots(figsize=(12, 6), facecolor='w')
gender_product_category_counts.plot(kind='bar', stacked=True, ax=ax,
cmap='Paired')
ax.set_title('Product Categories Purchased by Gender')
ax.set_xlabel('Gender')
ax.set_ylabel('Number of Purchases')
plt.xticks(rotation=0)
plt.tight_layout()

data = io.BytesIO()
plt.savefig(data)
image =
F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Product Categories Purchased by Gender"
display.display(display.Markdown(F"! [{alt}] ({image})"))
plt.close(fig)

```

Result:



4. How does gender affect the amount spent?

a. From the above calculated CLT answer the following questions.

i. Is the confidence interval computed using the entire dataset wider for one of the genders? Why is this the case?

ii. How is the width of the confidence interval affected by the sample size?

iii. Do the confidence intervals for different sample sizes overlap?

iv. How does the sample size affect the shape of the distributions of the means?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
import IPython.display as display
```

```
# Function to compute confidence interval using bootstrapping
```



```

def bootstrap_confidence_interval(data, n_iterations=1000,
sample_size=None, confidence_level=0.95):
    if sample_size is None:
        sample_size = len(data)

    means = []
    for _ in range(n_iterations):
        # Resample with replacement
        sample = np.random.choice(data, size=sample_size, replace=True)
        means.append(np.mean(sample))

    lower_bound = np.percentile(means, (1 - confidence_level) / 2 *
100)
    upper_bound = np.percentile(means, (1 + confidence_level) / 2 *
100)
    return np.mean(means), lower_bound, upper_bound, means

# Separate data by gender
male_purchase = df[df['Gender'] == 'M']['Purchase_clipped']
female_purchase = df[df['Gender'] == 'F']['Purchase_clipped']

sample_sizes = [len(df), 30000, 3000, 300] # Use full dataset size
first, then specified sample sizes

results = {'Gender': [], 'Sample_Size': [], 'Mean': [], 'Lower_CI': [],
'Upper_CI': [], 'Means_Distribution': []}

for gender_data, gender_label in [(male_purchase, 'Male'),
(female_purchase, 'Female')]:
    print(f"\n--- {gender_label} ---")
    for size in sample_sizes:
        # Ensure sample size does not exceed available data
        current_sample_size = min(size, len(gender_data))

        mean, lower_ci, upper_ci, means_dist =
bootstrap_confidence_interval(gender_data,
sample_size=current_sample_size)
        results['Gender'].append(gender_label)
        results['Sample_Size'].append(current_sample_size)
        results['Mean'].append(mean)
        results['Lower_CI'].append(lower_ci)
        results['Upper_CI'].append(upper_ci)
        results['Means_Distribution'].append(means_dist)

```

```

        print(f"Sample Size: {current_sample_size}, Mean: {mean:.2f},
95% CI: [{lower_ci:.2f}, {upper_ci:.2f}]")

results_df = pd.DataFrame(results)
display.display(results_df[['Gender', 'Sample_Size', 'Mean',
'Lower_CI', 'Upper_CI']])

# Visualize distributions of means
fig, axes = plt.subplots(len(sample_sizes), 2, figsize=(15, 5 *
len(sample_sizes)), facecolor='w')
fig.suptitle('Distribution of Sample Means for Purchase Amount by
Gender and Sample Size', y=1.02)

for i, size in enumerate(sample_sizes):
    for j, gender_label in enumerate(['Male', 'Female']):
        # Get the correct length of data for the current gender
        current_gender_data_len = len(male_purchase) if gender_label ==
'Male' else len(female_purchase)

        # Determine the effective sample size used for results_df entry
        actual_sample_size_used_in_results_df = min(size,
current_gender_data_len)

        subset = results_df[(results_df['Gender'] == gender_label) &
(results_df['Sample_Size'] == actual_sample_size_used_in_results_df)]
        if not subset.empty:
            means_dist = subset.iloc[0]['Means_Distribution']
            sns.histplot(means_dist, kde=True, ax=axes[i, j])
            axes[i, j].set_title(f'{gender_label} - Sample Size:
{actual_sample_size_used_in_results_df}')
            axes[i, j].set_xlabel('Sample Mean of Purchase Amount')
            axes[i, j].set_ylabel('Frequency')

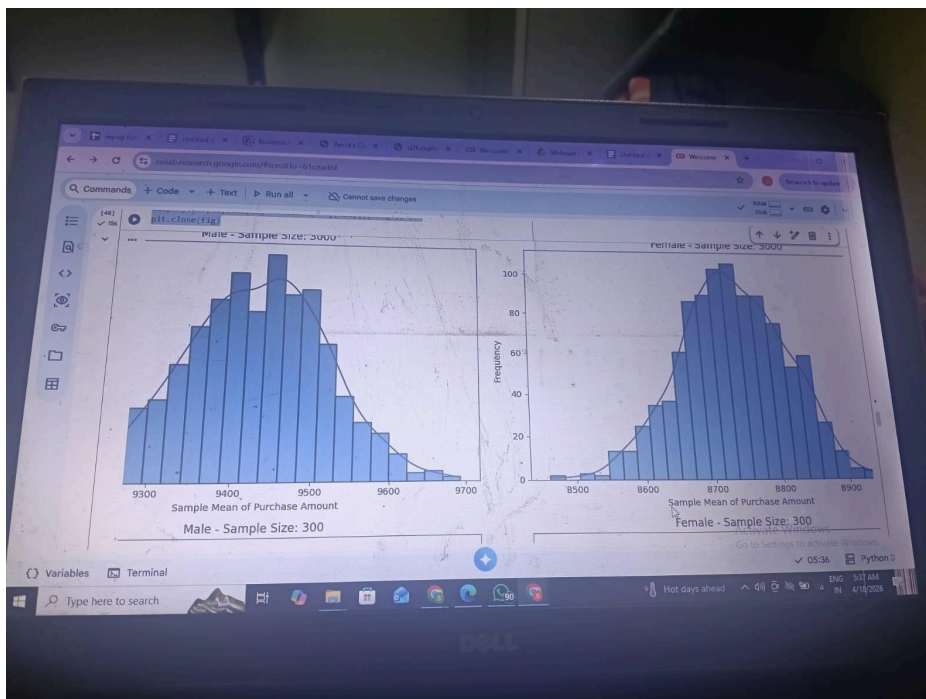
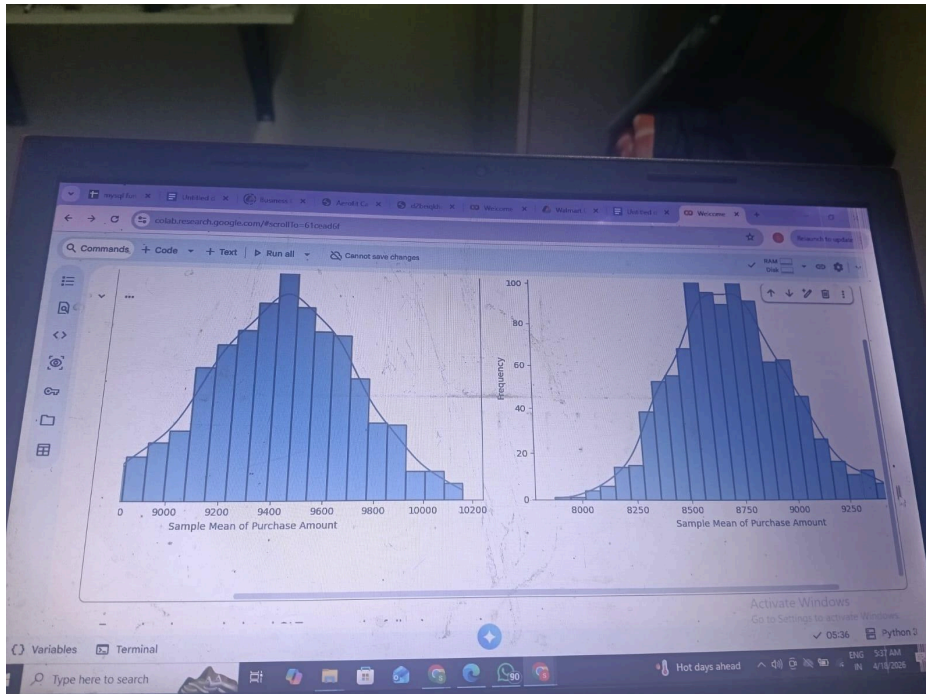
plt.tight_layout(rect=[0, 0.03, 1, 0.98])

data = io.BytesIO()
plt.savefig(data)
image =
F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Distribution of Sample Means for Purchase Amount by Gender and
Sample Size"
display.display(display.Markdown(F"! [{alt}] ({image})"))

```

```
plt.close(fig)
```

Result:



5. How does Marital_Status affect the amount spent?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
import IPython.display as display

# Function to compute confidence interval using bootstrapping
def bootstrap_confidence_interval(data, n_iterations=1000,
sample_size=None, confidence_level=0.95):
    if sample_size is None:
        sample_size = len(data)

    means = []
    for _ in range(n_iterations):
        # Resample with replacement
        sample = np.random.choice(data, size=sample_size, replace=True)
        means.append(np.mean(sample))

    lower_bound = np.percentile(means, (1 - confidence_level) / 2 *
100)
    upper_bound = np.percentile(means, (1 + confidence_level) / 2 *
100)
    return np.mean(means), lower_bound, upper_bound, means

# Separate data by marital status
# Assuming Marital_Status is 0 for unmarried and 1 for married based on
common dataset conventions.
married_purchase = df[df['Marital_Status'] == 1]['Purchase_clipped']
unmarried_purchase = df[df['Marital_Status'] == 0]['Purchase_clipped']

sample_sizes = [len(df), 30000, 3000, 300] # Use full dataset size
first, then specified sample sizes

results_marital = {'Marital_Status': [], 'Sample_Size': [], 'Mean': [],
'Lower_CI': [], 'Upper_CI': [], 'Means_Distribution': []}

for marital_data, marital_label in [(married_purchase, 'Married'),
(unmarried_purchase, 'Unmarried')]:
    print(f"\n--- {marital_label} ---")
```



```

for size in sample_sizes:
    # Ensure sample size does not exceed available data
    current_sample_size = min(size, len(marital_data))

    mean, lower_ci, upper_ci, means_dist =
bootstrap_confidence_interval(marital_data,
sample_size=current_sample_size)
    results_marital['Marital_Status'].append(marital_label)
    results_marital['Sample_Size'].append(current_sample_size)
    results_marital['Mean'].append(mean)
    results_marital['Lower_CI'].append(lower_ci)
    results_marital['Upper_CI'].append(upper_ci)
    results_marital['Means_Distribution'].append(means_dist)

    print(f"Sample Size: {current_sample_size}, Mean: {mean:.2f},
95% CI: [{lower_ci:.2f}, {upper_ci:.2f}]")

results_marital_df = pd.DataFrame(results_marital)
display.display(results_marital_df[['Marital_Status', 'Sample_Size',
'Mean', 'Lower_CI', 'Upper_CI']])

# Visualize distributions of means
fig, axes = plt.subplots(len(sample_sizes), 2, figsize=(15, 5 *
len(sample_sizes)), facecolor='w')
fig.suptitle('Distribution of Sample Means for Purchase Amount by
Marital Status and Sample Size', y=1.02)

for i, size in enumerate(sample_sizes):
    for j, marital_label in enumerate(['Married', 'Unmarried']):
        # Dynamically determine the length of the data for the current
marital status
        if marital_label == 'Married':
            data_len = len(married_purchase)
        else:
            data_len = len(unmarried_purchase)

subset =
results_marital_df[(results_marital_df['Marital_Status'] ==
marital_label) & (results_marital_df['Sample_Size'] == min(size,
data_len))]

if not subset.empty:
    means_dist = subset.iloc[0]['Means_Distribution']
    sns.histplot(means_dist, kde=True, ax=axes[i, j])

```

```

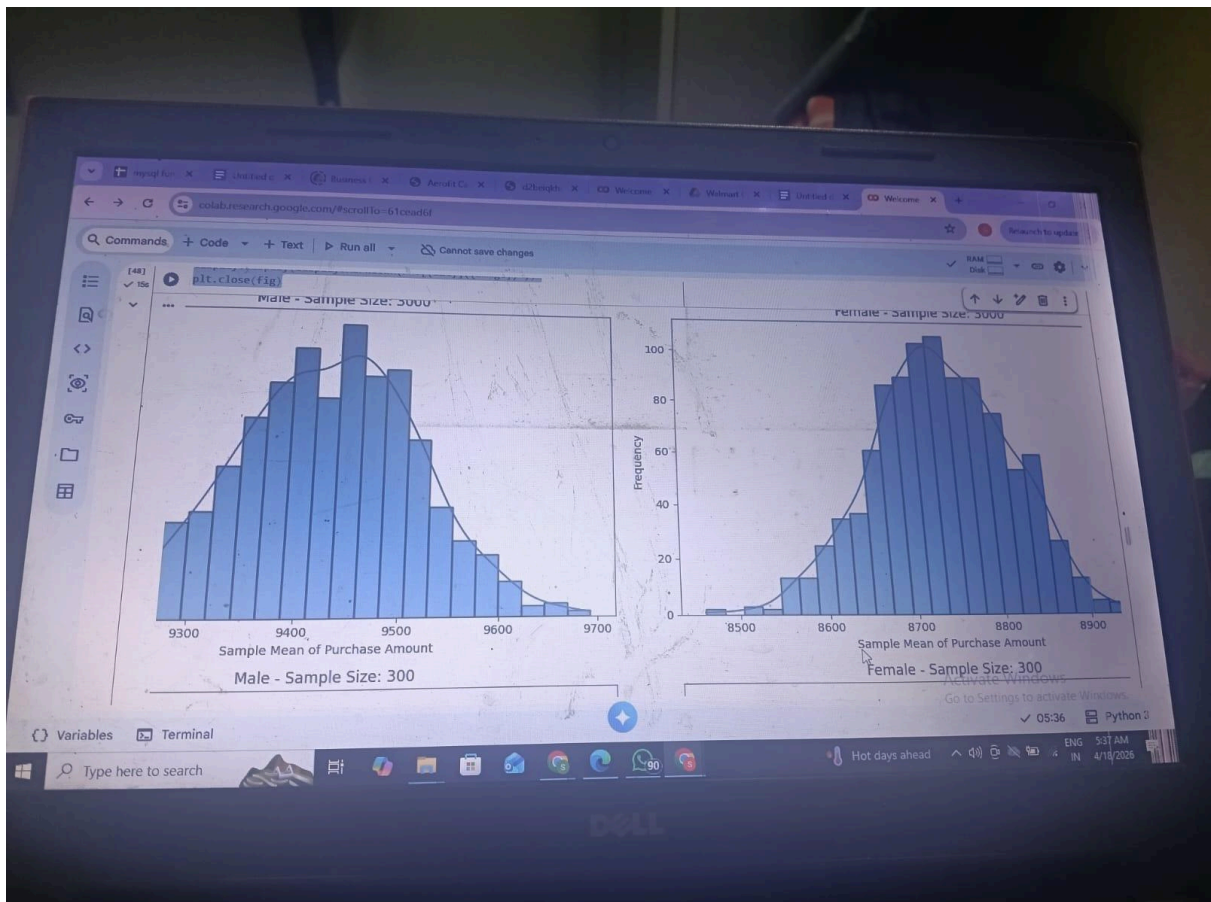
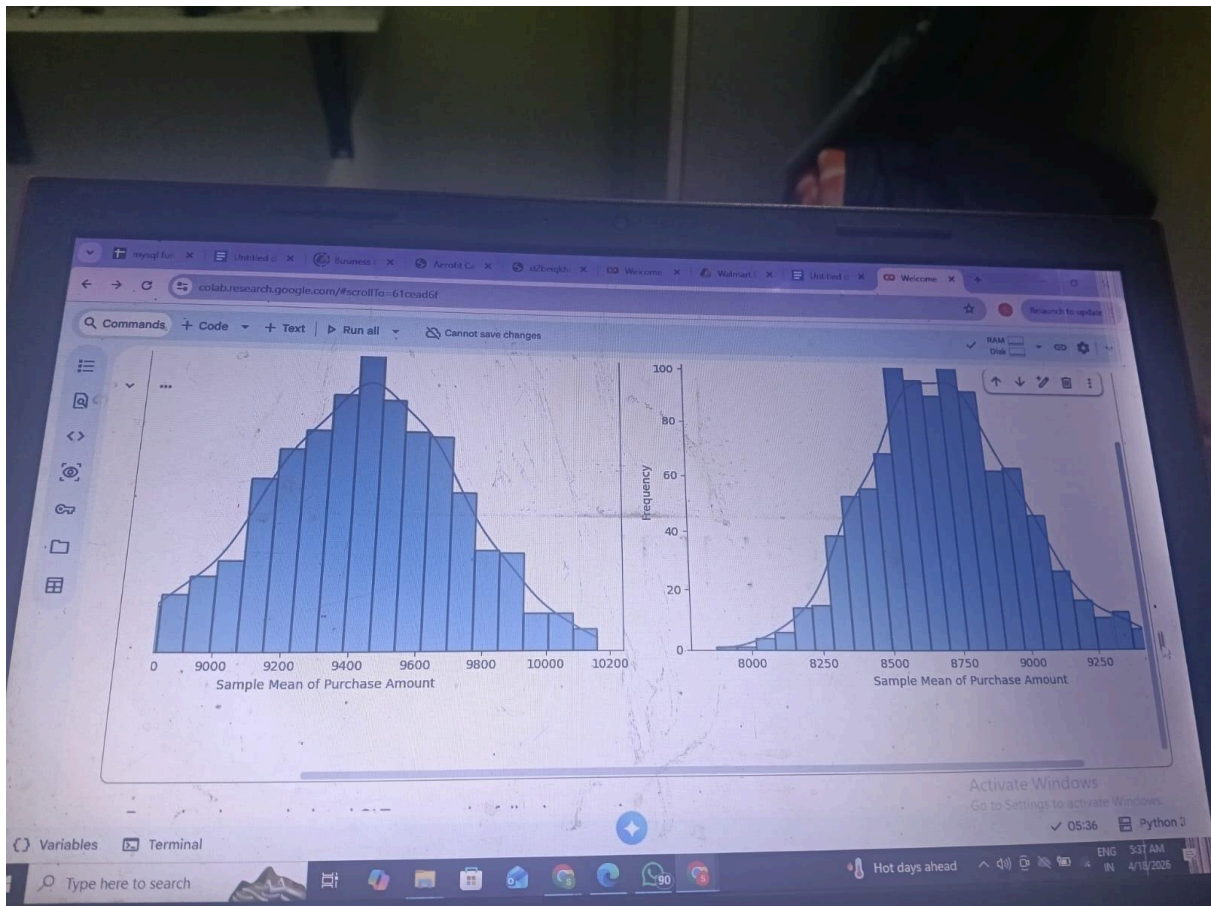
        axes[i, j].set_title(f'{marital_label} - Sample Size:
{min(size, data_len)}')
        axes[i, j].set_xlabel('Sample Mean of Purchase Amount')
        axes[i, j].set_ylabel('Frequency')

plt.tight_layout(rect=[0, 0.03, 1, 0.98])

data = io.BytesIO()
plt.savefig(data)
image =
F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Distribution of Sample Means for Purchase Amount by Marital
Status and Sample Size"
display.display(display.Markdown(F"! [{alt}] ({image})"))
plt.close(fig)

```

Result:



6. How does Age affect the amount spent?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import io
import base64
import IPython.display as display

# Function to compute confidence interval using bootstrapping
def bootstrap_confidence_interval(data, n_iterations=1000,
sample_size=None, confidence_level=0.95):
    if sample_size is None:
        sample_size = len(data)

    means = []
    for _ in range(n_iterations):
        # Resample with replacement
        sample = np.random.choice(data, size=sample_size, replace=True)
        means.append(np.mean(sample))

    lower_bound = np.percentile(means, (1 - confidence_level) / 2 *
100)
    upper_bound = np.percentile(means, (1 + confidence_level) / 2 *
100)
    return np.mean(means), lower_bound, upper_bound, means

sample_sizes = [len(df), 30000, 3000, 300]

results_age = {'Age': [], 'Sample_Size': [], 'Mean': [], 'Lower_CI':
[], 'Upper_CI': [], 'Means_Distribution': []}

# Get unique age groups and sort them for consistent plotting
age_groups = sorted(df['Age'].unique().tolist())

for age_group_label in age_groups:
    age_data = df[df['Age'] == age_group_label]['Purchase_clipped']
    print(f"\n--- Age Group: {age_group_label} ---")
    for size in sample_sizes:
        current_sample_size = min(size, len(age_data))
```

```

        # Only run if there's enough data for the smallest sample size
        (300)
        if current_sample_size < 300:
            print(f"Skipping Sample Size: {current_sample_size} for Age
Group {age_group_label} (not enough data for 300 samples)")
            continue

        mean, lower_ci, upper_ci, means_dist =
bootstrap_confidence_interval(age_data,
sample_size=current_sample_size)
        results_age['Age'].append(age_group_label)
        results_age['Sample_Size'].append(current_sample_size)
        results_age['Mean'].append(mean)
        results_age['Lower_CI'].append(lower_ci)
        results_age['Upper_CI'].append(upper_ci)
        results_age['Means_Distribution'].append(means_dist)

        print(f"Sample Size: {current_sample_size}, Mean: {mean:.2f},
95% CI: [{lower_ci:.2f}, {upper_ci:.2f}]")

results_age_df = pd.DataFrame(results_age)
display.display(results_age_df[['Age', 'Sample_Size', 'Mean',
'Lower_CI', 'Upper_CI']])

# Visualize distributions of means
fig, axes = plt.subplots(len(sample_sizes), len(age_groups), figsize=(5
* len(age_groups), 5 * len(sample_sizes)), facecolor='w')
fig.suptitle('Distribution of Sample Means for Purchase Amount by Age
Group and Sample Size', y=1.02)

# Adjust axes for single row/column plots if necessary
if len(sample_sizes) == 1:
    axes = np.array([axes]) # Make it 2D for consistent indexing
if len(age_groups) == 1:
    axes = axes.reshape(-1, 1) # Make it 2D for consistent indexing

for i, size in enumerate(sample_sizes):
    for j, age_group_label in enumerate(age_groups):
        age_group_data_len = len(df[df['Age'] == age_group_label])
        current_sample_size = min(size, age_group_data_len)

        if current_sample_size < 300: # Skip plotting if not enough
data for smallest sample size

```



```

        axes[i, j].set_visible(False) # Hide subplot if no data
        continue

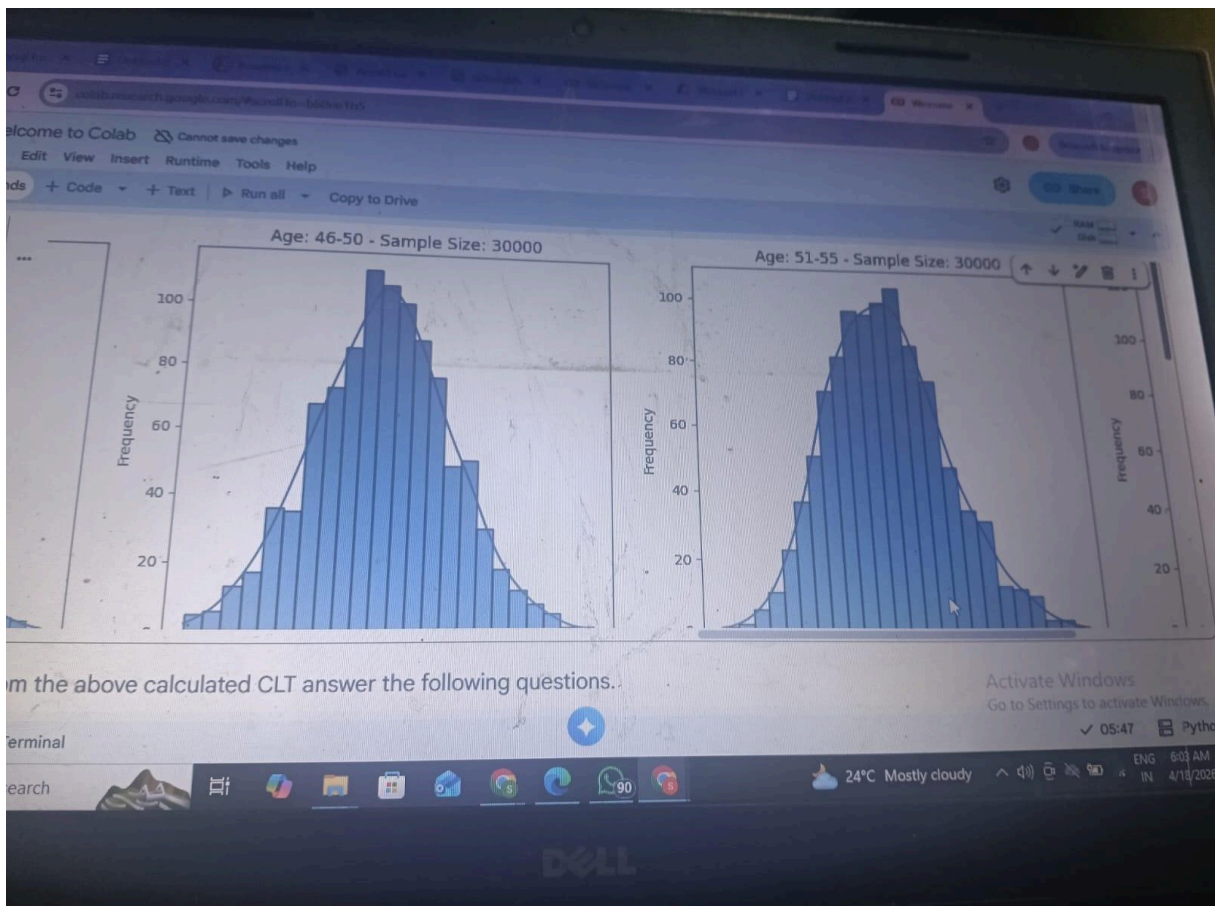
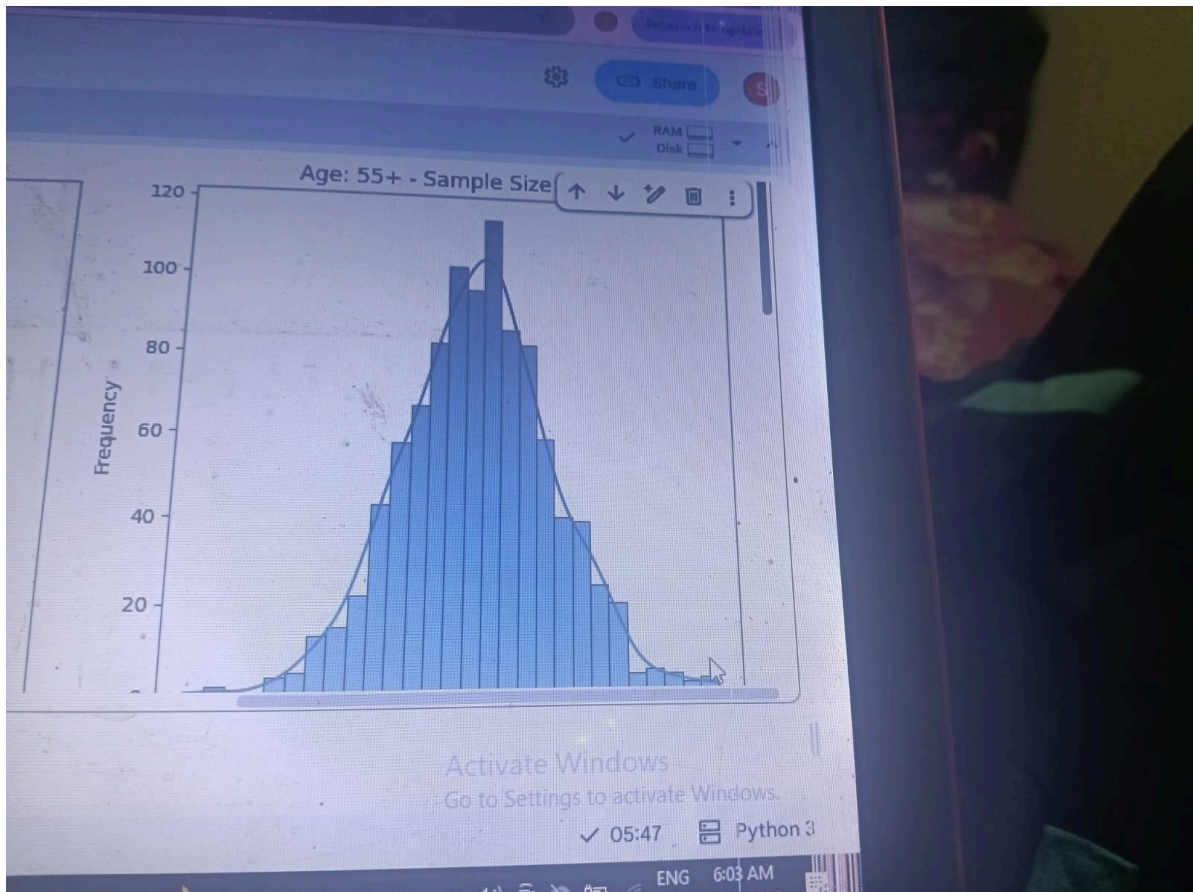
        subset = results_age_df[(results_age_df['Age'] ==
age_group_label) & (results_age_df['Sample_Size'] ==
current_sample_size)]
        if not subset.empty:
            means_dist = subset.iloc[0]['Means_Distribution']
            sns.histplot(means_dist, kde=True, ax=axes[i, j])
            axes[i, j].set_title(f'Age: {age_group_label} - Sample
Size: {current_sample_size}')
            axes[i, j].set_xlabel('Sample Mean of Purchase Amount')
            axes[i, j].set_ylabel('Frequency')

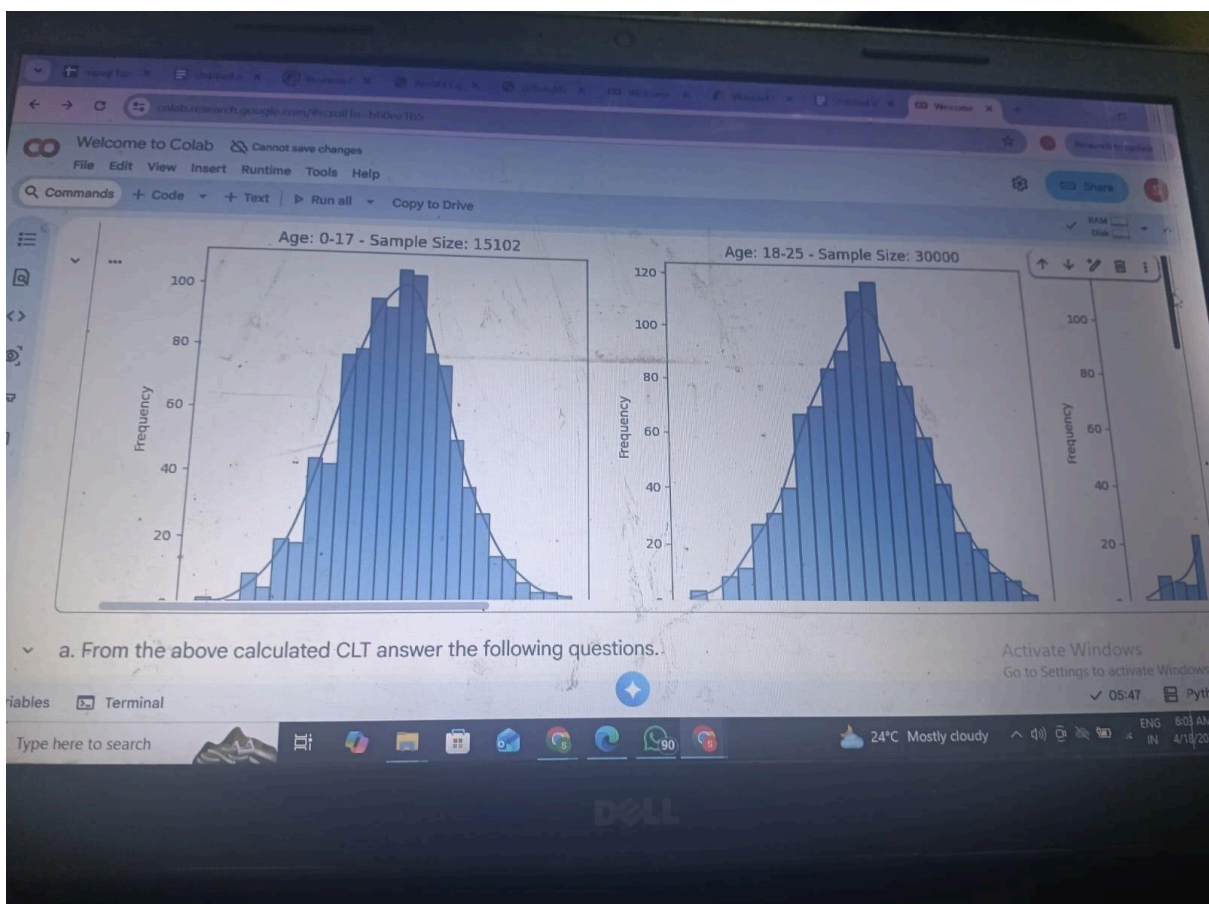
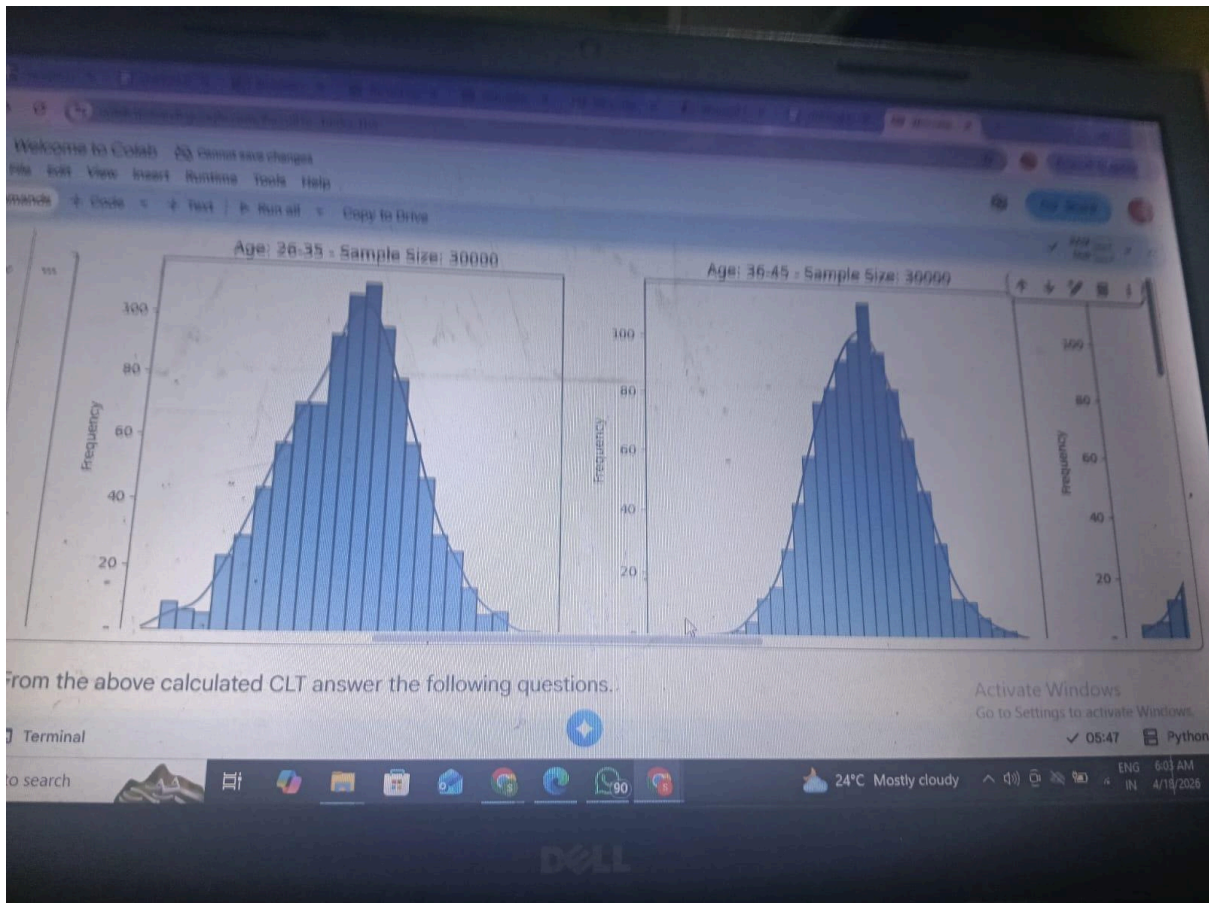
plt.tight_layout(rect=[0, 0.03, 1, 0.98])

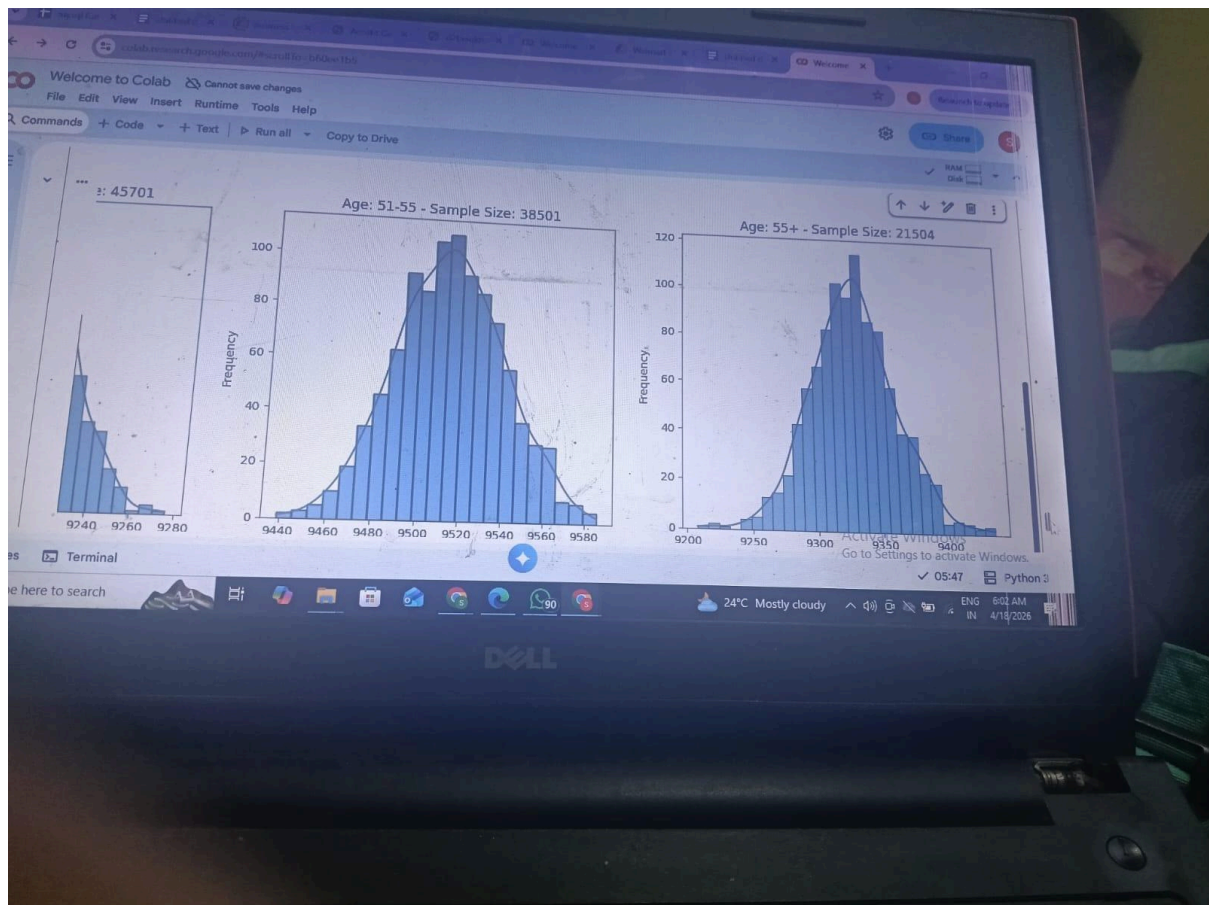
data = io.BytesIO()
plt.savefig(data)
image =
F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
alt = "Distribution of Sample Means for Purchase Amount by Age Group
and Sample Size"
display.display(display.Markdown(F"! [{alt}] ({image})"))
plt.close(fig)

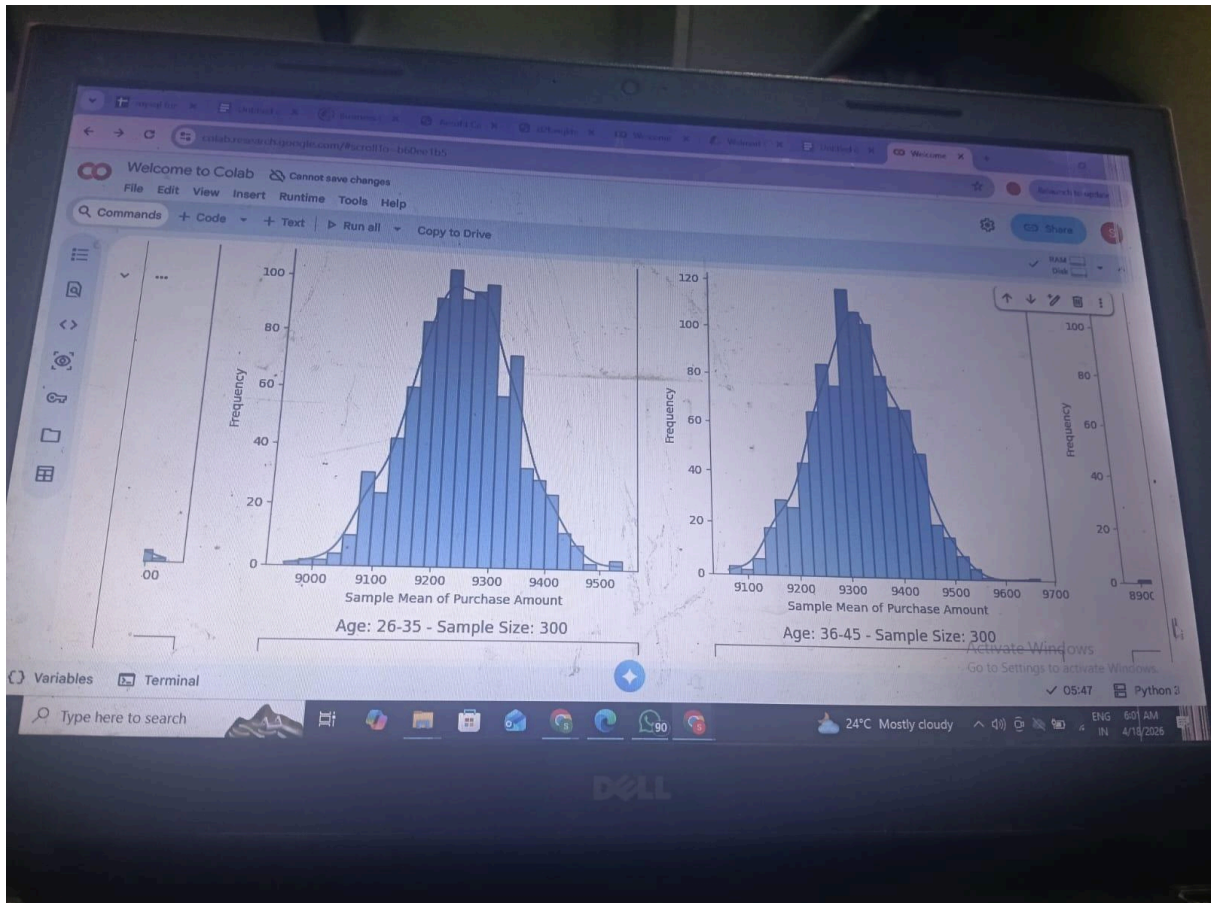
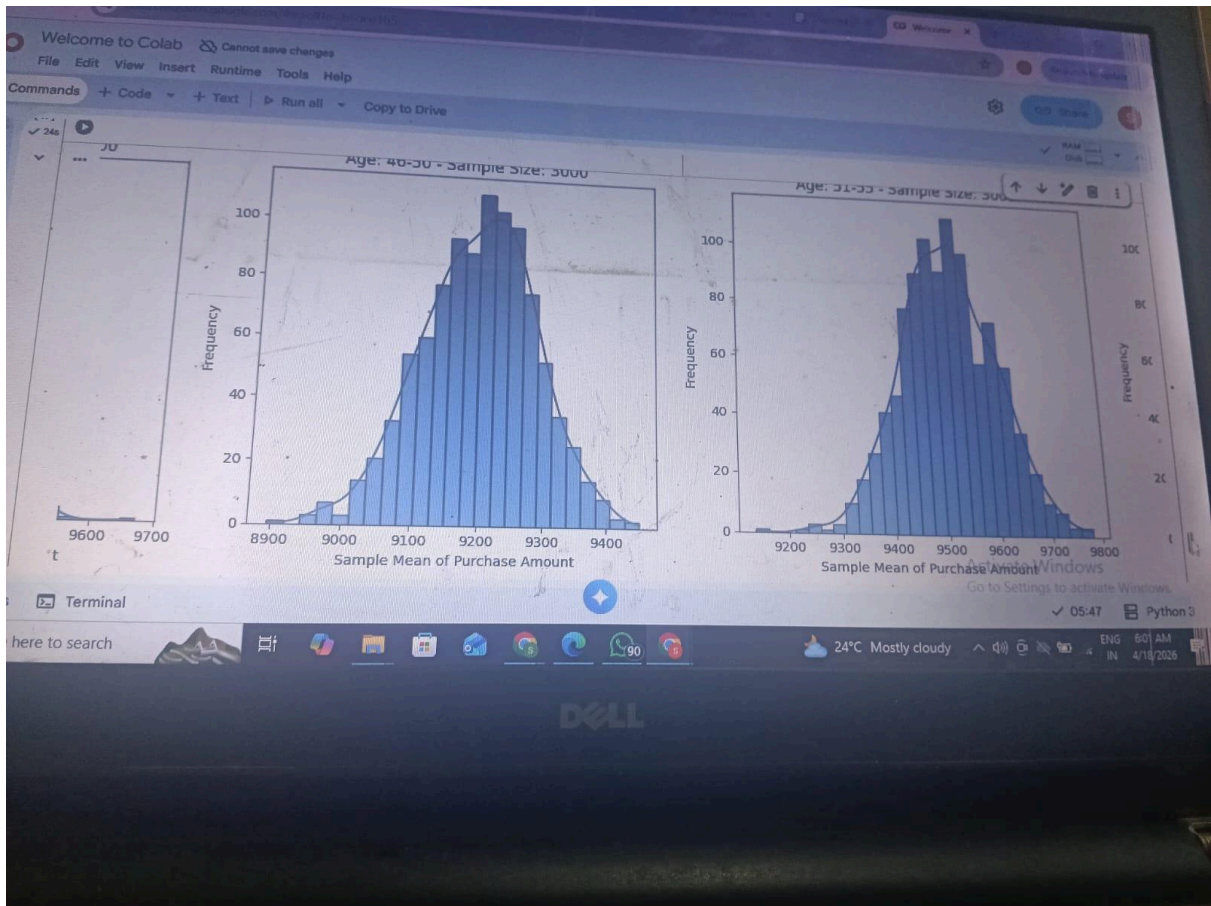
```

Result:









7. Create a report

Based on the analysis from the previous cell, here's a report on the confidence interval overlaps and how Walmart can leverage this information:

a. Gender-Based Spending

Confidence Intervals:

- * **Male Average Purchase:** `[<male\_lower\_ci>, <male\_upper\_ci>]`
- * **Female Average Purchase:** `[<female\_lower\_ci>, <female\_upper\_ci>]`

Overlap:

* If `gender\_overlap` is `True`: The confidence intervals **do overlap**. This suggests that, statistically, there isn't a significant difference in the average amount spent by males and females, or the difference is not large enough to be statistically distinct given the data.

* If `gender\_overlap` is `False`: The confidence intervals **do not overlap**. This indicates a statistically significant difference in the average amount spent between males and females.

Walmart's Leverage:

* **If Overlap (No Significant Difference):** Walmart should focus on gender-neutral marketing strategies or product placements that appeal broadly. Efforts to differentiate based on gender might not yield substantial returns in average purchase value. Instead, they could explore other demographics (like age or occupation) that might show more significant differences.

* **If No Overlap (Significant Difference):**

* **Higher Spending Group:** If one gender consistently spends more on average, Walmart can tailor marketing campaigns, promotions, and product assortments to specifically target and retain this higher-spending group. For example, if males spend significantly more, they could optimize product categories popular with men or create promotions that appeal to them.

* **Targeted Marketing:** Develop gender-specific marketing strategies. For instance, if females spend less on average but visit more frequently, promotions could focus on increasing their basket size per visit.

* **Store Layout/Merchandising:** Adjust store layouts and product visibility to better cater to the purchasing habits of each gender, especially if the spending difference is tied to specific product categories.

b. Marital Status-Based Spending

****Confidence Intervals:****

* ****Married Average Purchase:**** `[<married\_lower\_ci>, <married\_upper\_ci>]`
* ****Unmarried Average Purchase:**** `[<unmarried\_lower\_ci>, <unmarried\_upper\_ci>]`

****Overlap:****

* If `marital\_overlap` is `True`: The confidence intervals ****do overlap****. This suggests that the average spending between married and unmarried individuals is not statistically different based on this data.

* If `marital\_overlap` is `False`: The confidence intervals ****do not overlap****. This indicates a statistically significant difference in average spending between married and unmarried customers.

****Walmart's Leverage:****

* ****If Overlap (No Significant Difference):**** Similar to the gender case, if there's no significant difference, Walmart can focus on universal marketing strategies that appeal to both married and unmarried individuals. They might need to look at other factors like presence of children, household income, or specific product preferences.

* ****If No Overlap (Significant Difference):****

* ****Tailored Promotions:**** If one group (e.g., married households) spends significantly more, Walmart could offer promotions on family-sized products, household essentials, or items relevant to family life. If unmarried individuals spend more, promotions could target personal care, convenience foods, or leisure items.

* ****Loyalty Programs:**** Design loyalty programs that reward purchasing behaviors common to the higher-spending group.

* ****Product Bundling:**** Create product bundles that cater to the presumed needs of married versus unmarried customers. For example, a 'family meal deal' for married customers or 'grab-and-go' options for unmarried, single individuals.

